

SQL Server 인덱스 구조에 따른 성능 벤치마크 및 분석

Index Performance Compare in SQL Server

이현수

Software Engineering

kirukki050@gmail.com

2026년 8월 24일

Abstract—본 논문은 SQL Server 환경에서 힙(Heap), 비클러스터형 인덱스(Non-Clustered Index), 클러스터형 인덱스(Clustered Index) 구조 간의 CRUD 성능을 벤치마킹하고 그 결과를 분석한다. 흔히 알려진 통념인 “Non-Clustered Index의 CUD 연산 속도가 Clustered Index보다 더 빠르다”는 가설과 “데이터 변경 빈도가 높은 환경에서는 인덱스를 적용하면 안 된다”는 주장을 실제 데이터를 통해 수치적으로 검증하였다. 최대 100,000건의 데이터를 대상으로 한 실험 결과, Insert 연산에서는 인덱스 유지 오버헤드가 관찰되었으나 Update 및 Delete 연산에서는 인덱스 스캔을 통한 조건 탐색 성능 향상으로 인해 오히려 전체 소요 시간이 크게 단축됨을 확인하였다. 특히 단일 조회와 범위 조회 모두에서 인덱스가 비약적인 속도 향상을 가져왔음을 입증하였다.

Index Terms—SQL Server, Index, Performance Benchmark, Clustered Index, Non-Clustered Index, Heap, CRUD

I. 서론

데이터베이스 관리 시스템(DBMS)에서 인덱스(Index)는 검색 연산의 효율성을 극대화하기 위해 널리 사용되는 필수적인 자료구조이다. 하지만 인덱스를 생성하고 유지하는 과정은 데이터 생성(Create), 수정(Update), 삭제>Delete) 시에 추가적인 디스크 I/O와 페이지 분할(Page Split) 등의 오버헤드를 수반한다. 이로 인해 실무에서는 “데이터 변경 빈도가 잦은 테이블에는 인덱스 사용을 지양해야 한다”거나 “CUD 연산에는 Non-Clustered Index가 Clustered Index보다 유리하다”는 통념이 존재한다.

본 연구의 목적은 이러한 통념들이 실제 데이터베이스 환경(Microsoft SQL Server)에서 어느 정도의 유효성을 가지는지 정량적인 수치를 통해 팩트체크하는 데 있다. 이를 위해 Python 스크립트를 기반으로 데이터베이스에 100건부터 100,000건까지의 데이터를 적재하며, 힙(Pure Heap), 비클러스터형 인덱스를 포함한 힙(Heap + NC Index), 클러스터형 인덱스(Clustered Index) 등 세 가지 구조에서 CRUD 쿼리 성능을 정밀하게 측정 및 비교하였다.

II. 연구 방법 및 환경

A. 실험 환경

본 벤치마크는 다음과 같은 하드웨어 및 소프트웨어 환경에서 진행되었다.

- **OS:** Ubuntu 24.04 LTS
- **CPU:** Intel(R) Core(TM) i5-12400F (6 Cores / 12 Threads)

- **Memory:** 32 GB
- **Storage:** 120GB SSD / 1TB Samsung NVMe SSD
- **Database:** Microsoft SQL Server 2022 (Linux)

데이터베이스와의 연결은 Python 3.7+ 환경에서 pyodbc 라이브러리를 사용하였으며, 네트워크 통신 비용이 순수 데이터베이스 성능 측정에 미치는 영향을 최소화하기 위해 모든 DML 쿼리는 `cursor.executemany` 함수와 `fast_executemany=True` 옵션을 통해 일괄 처리(Batching)하였다. 더불어 트랜잭션 오버헤드를 제어하기 위해 수동 Commit을 적용하였다.

B. 측정 방법 및 지표

각 테이블 구조별로 다음 연산에 대한 총 실행 시간 및 평균 소요 시간을 측정하였다.

- 1) **Insert (생성):** 1부터 N 까지 순차적으로 증가하는 ID를 삽입하는 순차 삽입(Sequential Insert)과 순서를 무작위로 섞어 페이지 분할을 유도하는 무작위 삽입(Random Insert)으로 나누어 측정하였다.
- 2) **Select Point (단일 조회):** 전체 데이터 중 최대 100개의 임의의 ID를 추출하여 단일 레코드를 조회(WHERE id = ?)하고 그 평균 수행 시간을 산출하였다.
- 3) **Select Range (범위 조회):** 전체 데이터의 10% 범위에 해당하는 연속된 구간을 100회 무작위로 지정하여 조회(WHERE id BETWEEN ? AND ?)하였다.
- 4) **Update & Delete (수정 및 삭제):** 무작위로 100개의 ID를 추출해 일괄 변경 및 삭제하고 소요 시간을 측정하였다.

III. 이론적 성능 예측

데이터베이스 인덱스는 일반적으로 B-Tree(B-트리) 자료구조를 기반으로 구현된다[2]. 위키피디아에 잘 알려진 바와 같이, B-Tree는 트리의 균형을 유지함으로써 탐색(Search), 삽입(Insert), 삭제>Delete), 수정(Update) 연산 모두에 대해 평균적으로 및 최악의 경우에도 $O(\log n)$ 의 시간 복잡도를 보장한다. 따라서 클러스터형 인덱스(Clustered Index)와 비클러스터형 인덱스(Non-Clustered Index)가 적용된 테이블은 이러한 연산들에서 $O(\log n)$ 의 성능을 보일 것으로 예상된다.

반면 힙(Heap) 테이블은 순서가 보장되지 않는 일반적인 배열 형태의 구조이다. 순수 힙 테이블의 경우 새로운 데이터를 끝에 단순히 추가하기만 하면 되므로 삽입(Insert)

연산은 $O(1)$ 의 성능을 갖는다. 하지만 특정 데이터를 찾기 위한 검색, 그리고 대상을 찾아야만 수행 가능한 삭제 및 변경 연산의 경우 전체 데이터를 순차적으로 탐색(Table Scan)해야 하므로 $O(n)$ 의 시간 복잡도를 갖는다. 이러한 연산 비용 및 시간 복잡도에 대한 이론적 예측은 데이터베이스 시스템 교재인 *Database Management Systems*(Raghu Ramakrishnan, Johannes Gehrke 저, 3판)의 8.4.1 절 ‘Cost Model’을 참고하였다 [3].

IV. 실험 결과

100,000건의 데이터를 기준으로 측정한 주요 벤치마크 결과는 다음과 같다.

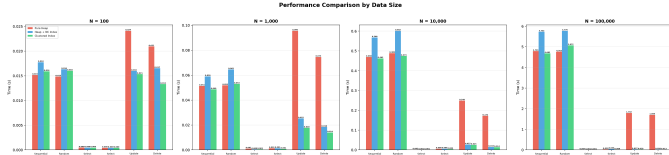


Fig. 1. 연산 종류에 따른 소요 시간 비교 (막대 그래프)

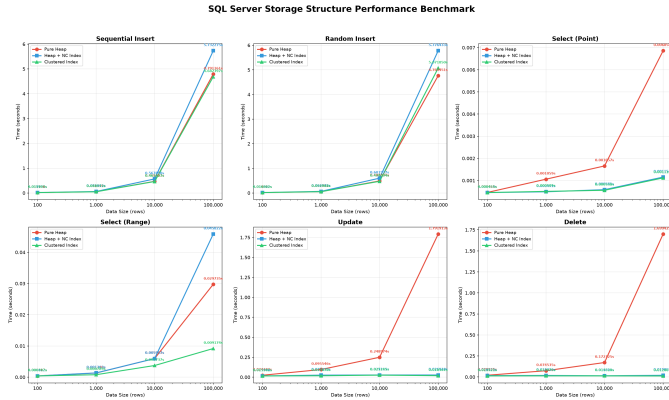


Fig. 2. 데이터 건수 증가에 따른 연산 소요 시간 변화 (선 그래프)

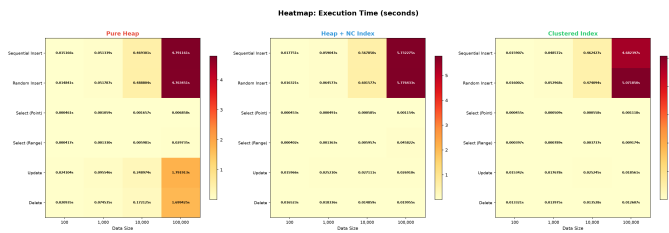


Fig. 3. 인덱스 구조와 연산 간의 성능 차이 (히트맵)

A. 조회 성능 (Select)

단일 조회(Select Point)의 경우, Clustered Index와 Heap + NC Index가 모두 평균 0.0011초의 소요 시간을 기록하여 Pure Heap(0.0068초) 대비 우수한 성능을 보였다. 범위 조회(Select Range)에서는 데이터가 물리적으로 정렬된 Clustered Index가 0.0091초로 가장 빠른 수행 시간을 기록했으며, 뒤이어 Pure Heap(0.0297초), Heap + NC Index(0.0458초) 순으로 나타났다.

B. 수정 및 삭제 성능 (Update & Delete)

조건부 쿼리를 수반하는 수정 및 삭제의 경우, 대상을 찾기 위한 탐색 과정이 병행된다. 실험 결과 Clustered Index와 Heap + NC Index는 모두 약 0.01초에서 0.02초의 짧은 소요 시간을 기록했다. 반면, 인덱스가 부재하여 전체 테이블 스캔(Table Scan)을 수행해야 하는 Pure Heap의 경우 약 1.7초가 소요되었다. 이를 통해 조건 탐색이 동반된 Update/Delete 연산에서는 인덱스 갱신 비용을 상쇄하고도 남을 만큼의 막대한 속도 향상을 얻을 수 있음을 확인하였다.

C. 삽입 성능 (Insert)

삽입 연산에서는 순차 삽입과 무작위 삽입 모두에서 Pure Heap과 Clustered Index가 약 4.6초에서 5.0초의 유사한 수행 시간을 기록하였다. 하지만 Heap + NC Index 구조의 경우 약 5.7초가 소요되어, 데이터 추가에 따른 인덱스 트리의 재정렬 비용이 존재함을 확인하였다.

V. 결론

본 연구를 통해 데이터베이스 인덱스에 대한 일반적인 통념들을 실증적으로 검증할 수 있었다. 첫째, 데이터 변경(CUD) 빈도가 높을수록 인덱스를 지양해야 한다는 통념은 절반만 타당하다. Insert에는 오버헤드가 발생하지만, 특정 행을 타겟팅하는 Update와 Delete 쿼리에서는 조건부 탐색이 필요하므로 오히려 인덱스의 유무가 성능 향상에 결정적인 기여를 한다. 둘째, Clustered Index는 CUD 전반에서 준수한 성능을 보여주었으며, 특히 광범위한 데이터를 읽어들이는 범위 조회(Range Scan)에서는 압도적인 우위를 점하였다. 결과적으로 인덱스 설계 시에는 단순히 변경의 빈도뿐만 아니라 연산의 구체적 형태(탐색을 수반하는가 여부)를 종합적으로 고려하여 아키텍처를 결정해야 한다.

REFERENCES

- [1] “Index Performance Compare Repository,” GitHub, 2026. [Online]. Available: Local Workspace (README.md, src/benchmark.py, result/).
- [2] Wikipedia, “B-tree,” [Online]. Available: <https://en.wikipedia.org/wiki/B-tree>.
- [3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003, sec. 8.4.1 Cost Model.